

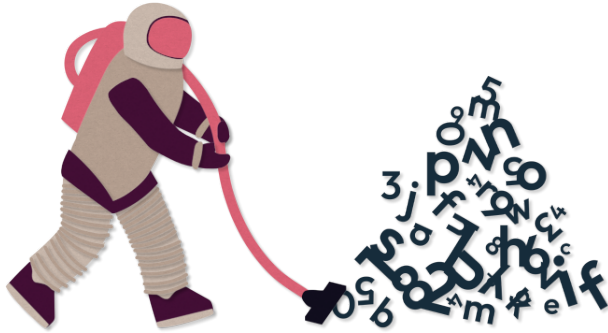
PROGRAMMING WITH PYTHON



Jumping
Rivers

CREATING CLARITY WITH YOUR DATA.

Saving organisations like yours thousands of work hours.



Jumping Rivers is an analytics company whose passion is data and machine learning. We help our clients move from data storage to data insights. Our trainers and consultants come with over 100 years combined experience in R, Python, Stan, Scala and other programming languages.

Consultancy

- Posit (formerly RStudio) certified - the only UK partner.
- We have plans for managing all Posit products. From on-demand support to full care packages.
- As a Posit reseller, we can provide free consultancy.
- Experts in both data science and data engineering.
- Creation of bespoke dashboards and Shiny apps.
- Optimisation of your algorithms.



Training

- R/Python/Stan/Scala
- The Tidyverse
- Machine Learning and Tensorflow
- Dashboards with Shiny
- Statistical Modelling
- And many more!



hello@jumpingrivers.com

@jumping_uk

“LET US CHANGE OUR TRADITIONAL ATTITUDE TO THE CONSTRUCTION OF PROGRAMS. INSTEAD OF IMAGINING THAT OUR MAIN TASK IS TO INSTRUCT A COMPUTER WHAT TO DO, LET US CONCENTRATE RATHER ON EXPLAINING TO HUMAN BEINGS WHAT WE WANT A COMPUTER TO DO.”

— DONALD KNUTH, *LITERATE PROGRAMMING*

Conditionals and Booleans

When programming we often want to *conditionally* execute certain bits of code. In this way we look to be able to change a programme's behaviour depending on the outcome of some tests.¹

For example, we may wish to:

- Compute the square root of a number, `if` that number is positive
- Save data to a file, `if` that file doesn't already exist
- Extract a row from a `DataFrame`, `if` the row has some certain quality

¹ In computing these "tests" are often referred to as conditionals.

0.1 Simple conditions

Before we can write programmes which conditionally execute code, we first need to learn how to write conditions (our tests).² To do so we will use the relation operators: `==`, `!=`, `<`, `<=`, `>`, and `>=`.

In the following chunks, suppose that `x = 5` and `y = 7`. To test for equality, we use the `==` operator:

```
# Does x equal 5?
x == 5
#> True
```

```
# Does x equal y?
x == y
#> False
```

To test if two values are *not* equal, we use the `!=` operator

```
# Does x not equal 5?
x != 5
#> False
```

```
# Does x not equal y?
x != y
#> True
```

We can also test for *inequality* with the greater than and less than symbols:

```
# Is x less than 6?
x < 6
#> True
```

```
# Is x less than or equal to y?
x <= y
```

² Whenever we ask Python a question (our conditions / tests), Python will always respond with either `True` or `False`.

Chunk 1: Similarly, we would test if `x` is greater than `6` with `x > 6`.

Chunk 2: Again, in a similar manner, we would test if `x` is greater than or equal to `y` with `x >= y`.

```
#> True
```

0.2 Multiple conditions

Quite often we'll want to test *more than one* condition. This is possible with the introduction of two new operators: `&` and `|`, these are the “and” and “or” operators, respectively.³

If we put an ampersand (`&`) between two conditions, we are testing if *both* conditions are `True`.

```
# Is both x greater than 0 and y greater than 10?
(x > 0) & (y > 10)
#> False
```

When combining multiple tests with `&` and `|` we isolate the individual tests with parentheses:

```
# Is both x greater than 0 *and* y less than 10?
(x > 0) & (y < 10)
#> True
```

If we put a pipe (`|`) between two conditions, we are testing if *either* of the conditions are `True`:

```
# Is either x greater than 0 *or* y greater than 10?
(x > 0) | (y > 10)
#> True
```

0.3 `if` statements

The `if` statement allows us to conditionally execute code.

The key elements in a Python `if` statement are:

- **The keyword `if`**: we use this to tell Python that we are about to create an `if` statement
- **A condition**: this is a test that will evaluate to `True` or `False`
- **The colon `:`**: this marks the end of the `if` header
- **The conditional code**: this is the code that is executed when our condition evaluates to `True`.⁴

0.4 Simple `if` statements

Let's create a simple `if` statement, using the condition `temp >= 18`:

```
temp = 20
if temp >= 18:
    print("Don the shorts.")
#> Don the shorts.
```

```
temp = 4
if temp >= 18:
    print("Don the shorts.")
```

We can also use conditionals with `&` and `|` in our `if` statements:

³ You may also see others use `and` and `or` operators in-place of `&` and `|`. For these simple examples the operators will behave identically. However, `and` and `or` don't play as nice with `np.ndarray`s and `pd.DataFrame`s as `&` and `|` do. As such, and to keep things simple, we'll just get into the habit of using `&` and `|` everywhere.

⁴ The conditional code body must be indented by 4 spaces with respect to its parent `if`.

Chunk 3: Although the conditional code to be executed here involved a call to `print()`, our code to be conditionally executed can be *any* valid Python code.

```
temp = 20
day = "Saturday"
if (temp >= 18) & (day == "Saturday"):
    print("Prepare the BBQ.")
#> Prepare the BBQ.
```

Chunk 4: To make our day comparison case insensitive we could have used the condition `day.lower() == "saturday"` .

0.5 The `else` statement

An `else` statement can be used to contain a block of code that should be executed if the `if` conditional evaluated to `False` . Again, as with `if` , the code associated with the `else` statement should be indented by four spaces:

```
temp = 20
if temp >= 18:
    print("Don the shorts.")
else:
    print("Maybe wear something full-length...")
#> Don the shorts.
```

Chunk 5: Python is a great tool to use to avoid wardrobe malfunctions.

```
temp = 4
if temp >= 18:
    print("Don the shorts.")
else:
    print("Maybe wear something full-length...")
#> Maybe wear something full-length...
```

0.6 The `elif` statement

The `elif` statement allows us to combine multiple checks one after another. Once an expression has been evaluated to `True` Python will evaluate that code block and skip to the end:

```
temp = 16
if temp >= 18:
    print("Don the shorts.")
elif temp >= 14:
    print("Are three-quarters still in fashion?")
else:
    print("Maybe wear something full-length...")
#> Are three-quarters still in fashion?
```

We can chain as many `elif` statements together as we like. Remember: once an expression has been evaluated to `True` Python will evaluate that code and skip to the end:

```
x = -1
if x < -10:
    print("x is teeny-tiny")
elif x < 0:
```

```

    print("x is between -10 and 0")
elif x < 10:
    print("x is between 0 and 10")
else:
    print("oh lawd he comin'")
#> x is between -10 and 0

```

0.7 Boolean indexing

We can use booleans (`True` and `False`) to extract data from `DataFrame`s. This idea becomes particularly powerful when we derive our booleans from a conditional test. In this way we will be able to *conditionally* extract data from a `DataFrame`.

Let's create a pandas `DataFrame` from a dictionary to play with:

```

import pandas as pd

the_north = {
    "city": ["Manchester", "York", "Leeds", "Birmingham"],
    "population": [553000, 210000, 789000, 1145000],
    "area": [115.6, 271.9, 551.7, 267.8]
}

the_north = pd.DataFrame(the_north)
the_north
#>      city  population   area
#> 0  Manchester    553000  115.6
#> 1      York     210000  271.9
#> 2     Leeds     789000  551.7
#> 3 Birmingham   1145000  267.8

```

We can then extract values from this with lists, arrays, or Series' of booleans:

```

mask = [True, False, False, True]
the_north.loc[mask]
#>      city  population   area
#> 0  Manchester    553000  115.6
#> 3  Birmingham   1145000  267.8

```

This has returned rows `0` and `3`; the positions in `mask` containing `True`. So far, so not that useful...

However, this idea becomes more powerful when we construct our `True` and `False` values from the output of a test:

```

# Construct boolean mask
mask = the_north["population"] > 600000
mask
#> 0    False
#> 1    False
#> 2     True
#> 3     True

```

```
#> Name: population, dtype: bool
```

We can then use `mask` to extract all the rows of `the_north` with `"population"`'s greater than `600000`:

```
the_north.loc[mask]
#>      city  population  area
#> 2    Leeds      789000  551.7
#> 3 Birmingham     1145000  267.8
```

Here, we could have achieved the above two step process in a single line of code:

```
# Extract rows of the_north with population > 600000
the_north.loc[the_north["population"] > 600000]
#>      city  population  area
#> 2    Leeds      789000  551.7
#> 3 Birmingham     1145000  267.8
```

We can perform more complex conditional extracting with `&` and `|`:

```
mask = ((the_north["population"] > 500000)
        & (the_north["area"] < 300))
the_north.loc[mask]
#>      city  population  area
#> 0 Manchester     553000  115.6
#> 3 Birmingham     1145000  267.8
```


Loops

In programming, it is common for us to wish to perform some tasks repeatedly. We can accomplish such tasks using *loops*.

For example, we may wish to:

- Perform the same analysis on multiple different datasets
- Perform some operation on each row of a DataFrame
- Load all the files in a folder with a particular file extension

Here we will see three different ways of constructing loops in Python.

1.1 `for` loops

We can use `for` loops to iterate over objects such as lists, strings, and arrays.

The key elements in a Python for loop are:

- **The keyword `for`** : we use this to tell Python that we are about to write a `for` loop
- **The loop variable**: this will be the value which changes at each iteration of our loop
- **The `in` keyword**: this separates our loop variable from our iterable
- **The iterable**: this is the object which we shall “iterate” over. It is often a list, string, or generator (more on these later)
- **The colon `:`** : this marks the end of the for loop header
- **The loop body**: this is the code that is executed at every iteration of our loop. Our loop body must be indented by 4 spaces

`for` loop example

We can construct a simple for loop as:

```
for i in [5, 6, 7, 8]:  
    print(i)
```

In Chunk 1.1 the loop variable was `i` and the iterable was the list `[5, 6, 7, 8]`. The loop body (the code to be executed at each iteration of the loop) contained only the code `print(i)`.

The code in Chunk 1.1 is equivalent to the more tediously typed:

```
print(5)  
print(6)
```

Chunk 1.1: A simple for loop.

```
print(7)
print(8)
```

Here we only looped over a list with 4 values. But what if we wanted to loop over lots more values? Would we have to type all these values out individually? Fortunately not. Python's `range()` can help us here:

```
for j in range(10):
    print(j)
```

In Chunk 1.2 the loop variable was `j` and the iterable was `range(10)`. Again, the loop body contained only the code `print(j)`. The code in Chunk 1.2 is equivalent to the rather cumbersome:

```
print(0)
print(1)
print(2)
print(3)
print(4)
print(5)
print(6)
print(7)
print(8)
print(9)
```

`range()` has the same `start`, `stop` pattern that we saw with list slicing:

```
for x in range(2, 6):
    print(x)
```

Here our loop variable was `x` and our iterable was `range(2, 6)`. This loop was then equivalent to the following code:

```
print(2)
print(3)
print(4)
print(5)
```

Chunk 1.2: Using `range()` to loop over the numbers from 0 to 9.

1.2 `while` loops

The `while` loop provides another way to construct loops in Python. With the `while` loop we execute a set of statements so long as a condition is `True`.

The key elements in a Python while loop are:

- The keyword `while`: we use this to tell Python that we are about to write a `while` loop
- The condition: something which evaluates to `True` or `False`
- The colon `:`: this marks the end of the `while` loop header
- The loop body: this is the code that is executed at every iteration of our loop. Our loop body must be indented by 4 spaces

`while` loop example

We can construct a simple `while` loop as:

```
i = 0
while i < 10:
    print(i)
    i = i + 1
```

In this example our condition was `i < 10`, and our loop body printed and then incremented `i`.

Let's consider another example, in which we print all the `i`s which when squared, are less than `70`:

```
i = 0
while i**2 < 70:
    print(i)
    i = i + 1
```

Aside: the `+=` syntax

In Python `i += 1` can be used as a shorthand for `i = i + 1`. We could use this shorthand in our `while` loop examples as:

```
i = 0
while i < 10:
    print(i)
    i += 1
```

1.3 Counting iterations with `enumerate()`

Sometimes we may wish to loop over some values, and have a counter which represents our progress through these values. `enumerate()` constructs an iterable with this behaviour:

```
seasons = ["Spring", "Summer", "Autumn", "Winter"]
for count, value in enumerate(seasons):
    print(count, value)
#> 0 Spring
#> 1 Summer
#> 2 Autumn
#> 3 Winter
```

1.4 Combining iterables with `zip()`

We can use `zip()` to loop over two (or more) iterables at once:

```
first_names = ["Regina", "Cady", "Ms."]
last_names = ["George", "Heron", "Norbury"]

for first, last in zip(first_names, last_names):
    print(f"{first} {last}")
#> Regina George
```

Chunk 1.3: Here we use Python's F-strings to concatenate the `first` and `last` loop variables.

#> Cady Heron
#> Ms. Norbury

Functions

A *function* is a re-usable block of code which is only run when we call it. Functions are a really great way to encapsulate tasks which we perform regularly. As our programmes get longer and become more complex, using functions can help keep our code tidy and maintainable.

2.1 Defining functions

The key elements in a Python function definition are:

- The keyword `def`: we use this to tell Python that we are about to `def`ine a function
- The function name: this allows us to refer back to our function later
- Parentheses: this is where we specify the arguments (or lack thereof) which our function can expect to receive
- The colon `:`: this marks the end of the function header
- The function body: this is the code that is executed whenever we call our function. Our function body must be indented by 4 spaces

2.2 Simple functions

Let's see a simple function definition:

```
def hello(person):  
    print("Hello " + person + ", it's me: your computer.")
```

We can *call* this function by its name: `hello`. To this function we must pass through a value to be interpreted as the `person` variable:

```
hello("Paul")  
#> Hello Paul, it's me: your computer.
```

```
hello("John")  
#> Hello John, it's me: your computer.
```

2.3 `return`ing values

Our functions may optionally *return* values. We do so with the `return` keyword. Let's define a new function called `compute_square` which takes a single argument and `return`s the square of the argument passed:

```
# Define a function with a return statement
def compute_square(x):
    return x ** 2
```

We can then call our function as:

```
compute_square(12)
#> 144
```

As our function is now `return` ing a value, we can assign the `return` ed value to a variable for re-use later:

```
squared = compute_square(7)
squared
#> 49
```

`return` ing multiple values

Our functions may also return *multiple* values. We specify all the objects we'd like to return, separated by commas, on the return line:

```
def test_cases(string):
    return string.lower(), string.upper(), string.title()
```

Our function will now return a *tuple* with three entries. Although we've not been formally introduced to the tuple before, it is similar to the familiar list object. The difference is that the tuple is *immutable* (lists are *mutable*) and so we *can't* change the values stored in a tuple, unlike how we can with lists.

```
cases = test_cases("All type no face")
type(cases)
#> <class 'tuple'>
```

We can extract values from our tuple just as we can with a list:

```
print(cases[0])
#> all type no face
print(cases[1])
#> ALL TYPE NO FACE
print(cases[2])
#> All Type No Face
```

We can also “unpack” these values when we call our function:

```
lower, upper, title = test_cases("hello, goodbye")
print(lower)
#> hello, goodbye
print(upper)
#> HELLO, GOODBYE
print(title)
#> Hello, Goodbye
```

2.4 Multiple Arguments

Our functions can take *multiple* arguments. When we specify multiple arguments we must separate them by commas:

```
def divider(x, y):
    return x / y
```

We can call such as function as:

```
divider(22, 7)
#> 3.142857142857143
```

The *order* in which we pass through our values is important. The first value which we pass to our function `divider()` will be interpreted as the argument `x`. The second value which we pass will be interpreted by our function as the argument `y`:

```
divider(7, 22)
#> 0.3181818181818182
```

Rather than implicitly relying on the order of the arguments, we may sometimes explicitly specify which values would be interpreted as which variables. In doing so we can pass our values through in any order we'd like. Consider:

```
# Interpret args by order
print(divider(7, 22))
#> 0.3181818181818182

# Explicitly specifying values for each arg
print(divider(y=7, x=22))
#> 3.142857142857143
```

2.5 No arguments

It is also possible to write functions which don't take *any* arguments:

```
def gimme5():
    return 5

gimme5()
#> 5
```

Although functions taking no arguments may not seem of obvious use, I use them often. For example, I often have a `load_data()` function (or some variation on this), which takes no arguments and returns some data associated with my project.

2.6 Default arguments

We can also construct functions which take *default* arguments. Default arguments are used when no value is given for a variable:

```
def weather_forecaster(weather="rain"):
    return "Tomorrow will be: " + weather
```

If we call this function *without* passing a value through for `weather`, our function will use the default value:

```
weather_forecaster()
#> 'Tomorrow will be: rain'
```

However, we can “override” default arguments by passing a value through:

```
weather_forecaster("grey")
#> 'Tomorrow will be: grey'
```

Multiple arguments with defaults

If we wish to define a function which takes *multiple arguments*, some of which are *optional*, we must specify the default arguments last:

```
# n takes a default argument and so
# must be specified after x
def to_the_power(x, n=2):
    return x ** n
```

We call this function as usual:

```
# No value for n. Function will use default
to_the_power(4)
#> 16

# Value passed through for n, will override default
to_the_power(4, 3)
#> 64
```

2.7 Good practices

Great, so we can write functions. But can we write functions which are well-documented, intuitive, and easy to (re-)use? I promise, although your function may make perfect to your present self, it often makes a *lot less sense* to your future self. (Speaking from many experiences of being sabotaged by my past self.) Let’s see a couple of tips for writing better functions.

Docstrings

We use *docstrings* to add documentation to our functions that can then be accessed from the `help()` function. Docstrings must be specified at the top of our function’s code block, and be surrounded by triple quotation marks:

```
def to_the_power(x, n=2):
    """This is a docstring."""
    return x ** n

help(to_the_power)
#> Help on function to_the_power in module __main__:
#>
#> to_the_power(x, n=2)
#>     This is a docstring.
```


Docstrings are useful not just for other people who may read your code (colleagues, friends, enemies, etc.), but for your future self. It is good practice to use your docstring to give a quick description of your function's purpose, and to detail and arguments that your function takes, and what types they should be.

The docstrings of the functions of the numpy package are a gold-standard to aim for:

```
import numpy as np
help(np.sum)
```

Let's give our function a numpy-style docstring:

```
def to_the_power(x, n=2):
    """
    Raise x to the power of n

    Parameters
    -----
    x : float, int, or np.ndarray
        The base value to exponentiate.
    n : float, int, optional
        The exponent to raise to. The default is 2.

    Returns
    -----
    float, int or np.ndarray
        x raised to the power of n.
    """
    return x ** n

help(to_the_power)
#> Help on function to_the_power in module __main__:
#>
#> to_the_power(x, n=2)
#>     Raise x to the power of n
#>
#>     Parameters
#>     -----
#>     x : float, int, or np.ndarray
#>         The base value to exponentiate.
#>     n : float, int, optional
#>         The exponent to raise to. The default is 2.
#>
#>     Returns
#>     -----
#>     float, int or np.ndarray
#>         x raised to the power of n.
```

Aside: another way of accessing help

Although we've previously used the `help()` function to view documentation, most modern Python IDEs allow a more convenient syntax using `?`:

```
to_the_power?
```

Using `?` formats documentation nicer than `help()` does. We can also use `??` to view the source code of functions:

```
to_the_power??
```

However, `??` won't reveal a lot for functions which in turn call C code (like much of the numpy package):

```
np.sum??
```

Naming

Try to choose function names which are indicative of your function's purpose.

```
# Bad function name
def my_func(x):
    """Compute the arithmetic mean of the list x."""
    return sum(x) / len(x)

my_func([57, 49, 27, 35])
#> 42.0
```

```
# Better function name
def compute_mean(x):
    """Compute the arithmetic mean of the list x."""
    return sum(x) / len(x)

compute_mean([57, 49, 27, 35])
#> 42.0
```

Comprehensions

Comprehensions are great for building new data structures quickly and with minimal code.

3.1 List comprehensions

List comprehensions provide a convenient way of quickly and efficiently building lists.

Multiplying values in a list

Let's imagine that we have some list `x = [1, 2, 3, 4, 5]`, and that we wish to multiply each entry in `x` by `2` *without* using any additional packages. We *could* solve this problem with a `for` loop as:

```
x = [1, 2, 3, 4, 5]

# I'm an empty list
y = []
for i in x:
    # Append 2 * i to our list y
    y.append(2 * i)

y
#> [2, 4, 6, 8, 10]
```

Alternatively, we could have solved this problem with a list comprehension:

```
z = [2 * i for i in x]
z
#> [2, 4, 6, 8, 10]
```

The syntax of the list comprehension follows the pattern:

```
[do_something for value in iterable]
```

Upper-casing strings in a list

Suppose we have a list of strings where each string is in lowercase. We would like to create a new list, with the strings from the original list upper-cased. This problem could be solved with a `for` loop:

```

pudding = ["how", "can", "you", "have", "any", "pudding",
           "if", "you", "don't", "eat", "your", "meat?"]

PUDDING = []
for word in pudding:
    PUDDING.append(word.upper())

PUDDING
#> ['HOW', 'CAN', 'YOU', 'HAVE', 'ANY', 'PUDDING', 'IF',
#> 'YOU', "DON'T", 'EAT', 'YOUR', 'MEAT?']

```

Alternatively, using our new-found comprehension skills, we could have solved this problem as:

```

# [do_something for value in iterable]
PUDDING = [word.upper() for word in pudding]
PUDDING
#> ['HOW', 'CAN', 'YOU', 'HAVE', 'ANY', 'PUDDING', 'IF',
#> 'YOU', "DON'T", 'EAT', 'YOUR', 'MEAT?']

```

if statements within comprehensions

Suppose we have a list of numbers:

```
numbers = [1, 24, 35, 67, 81, 2, 45, 89, 12]
```

and we want to construct a new list `odd_numbers`, containing (you guessed it) only the *odd* entries of `numbers`. We could have solved this problem with a `for` loop as:

```

numbers = [1, 24, 35, 67, 81, 2, 45, 89, 12]

odd_numbers = []
for num in numbers:
    # If a number has remainder 1 on division by 2 then it
    # must be odd
    if num % 2 == 1:
        odd_numbers.append(num)

odd_numbers
#> [1, 35, 67, 81, 45, 89]

```

We could also solve this problem with a list comprehension by noting that the simpler comprehension pattern seen previously can be extended to:

```
[do_something for value in iterable if condition]
```

To create our list `odd_numbers` we could use this pattern as:

```

odd_numbers = [num for num in numbers if num % 2 == 1]
odd_numbers
#> [1, 35, 67, 81, 45, 89]

```

3.2 Dictionary comprehensions

Dictionary comprehensions are *similar* to list comprehensions: providing a concise syntax to quickly construct dictionaries.

Let's suppose my granny gave me a cake recipe, and further, let's suppose that my granny gave me the ingredients for this recipe as a Python dictionary...

```
cake_ounces = {
    "sugar": 7,
    "butter": 7,
    "eggs": 4,
    "self-raising flour": 7
}
```

Unfortunately, the ingredients are measured in some old-fashioned measurement system (ounces??) which I'm not familiar with. The reliable source of information which is the internet says that there are [28.35](#) grams in [1](#) ounce. Let's use a for loop to create a dictionary detailing our cake's ingredients in grams:

```
# Create an empty dictionary
cake_grams = {}

# We can loop over key-value pairs with the .items() method
for ingredient, weight in cake_ounces.items():
    cake_grams[ingredient] = weight * 28.35 // 1

cake_grams
#> {'sugar': 198.0, 'butter': 198.0, 'eggs': 113.0, 'self-
#> raising flour': 198.0}
```

Alternatively we could have used a dictionary comprehension to solve this problem as:

```
cake_grams = {
    ingredient: weight * 28.35 // 1
    for ingredient, weight in cake_ounces.items()
}

cake_grams
#> {'sugar': 198.0, 'butter': 198.0, 'eggs': 113.0, 'self-
#> raising flour': 198.0}
```

The general syntax for a dictionary comprehensions is

```
new_dict = {key: do_something for value in iterable}
```

After making this cake I noted that it tasted a *little* too eggy for my palette. On reviewing my conversions I see that I shouldn't have multiplied the number of eggs by [28.35](#)

Amending our `for`-loop solution from above:

```

cake_grams = {}
for ingredient, weight in cake_ounces.items():
    # The eggs weren't measured in ounces so steer clear
    if ingredient != "eggs":
        cake_grams[ingredient] = weight * 28.35 // 1

cake_grams["eggs"] = cake_ounces["eggs"]

cake_grams
#> {'sugar': 198.0, 'butter': 198.0, 'self-raising flour':
#> 198.0, 'eggs': 4}

```

As with the list comprehension, our dictionary comprehension can be extended to include conditionals:

```

new_dict = {
    key: do_something for value in iterable if condition
}

```

With this we could have solved our recipe conversion mix-up with a dictionary comprehension as:

```

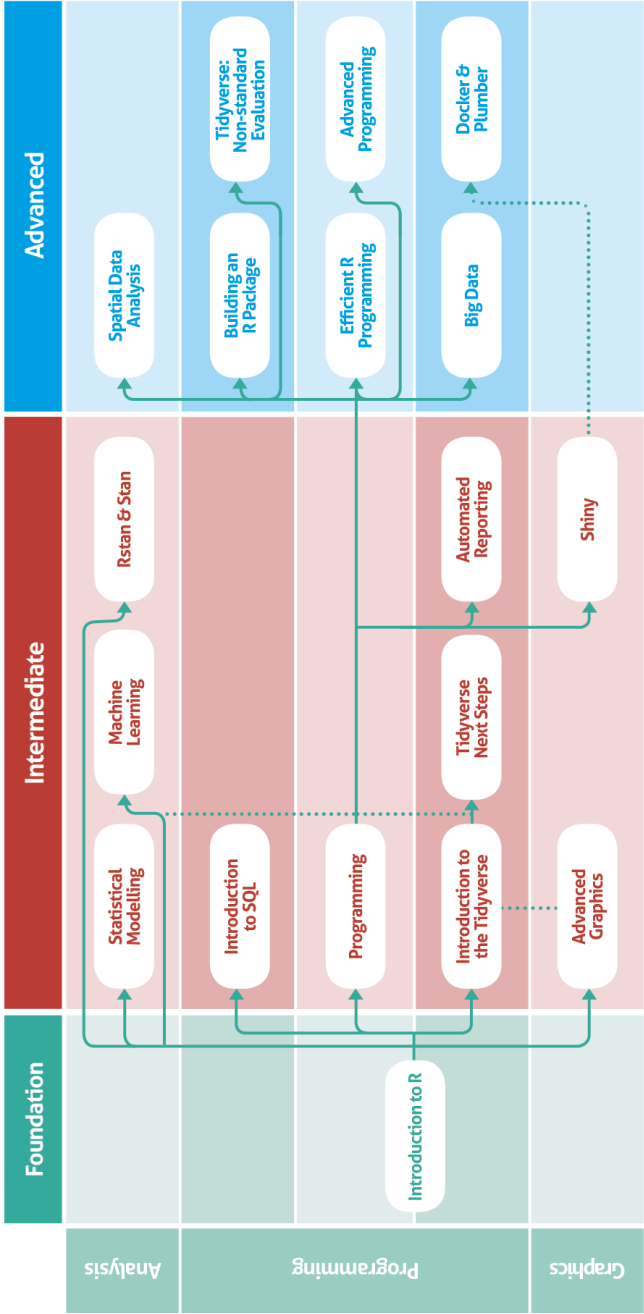
cake_grams = {ingredient: weight * 28.35 // 1
               for ingredient, weight in cake_ounces.items()
               if ingredient != "eggs"}

cake_grams["eggs"] = cake_ounces["eggs"]

cake_grams
#> {'sugar': 198.0, 'butter': 198.0, 'self-raising flour':
#> 198.0, 'eggs': 4}

```


Training Course Dependencies





OUR PARTNERS